



Figure 1: A shell acts as an interface between the user and the OS
(Image credits: <http://www.guru99.com/>)

In different operating systems, shell scripts are referred to by different names such as batch files (OS/2, MSDoS-Win95 stream), command procedures (VMS), and shell scripts (Windows NT stream and third-party derivatives). Mainframe operating systems are associated with a different set of terms.

Shell scripts help us to program different commands in chains and have the system execute them as a scripted event—the same as batch files. They also allow us to perform far more useful functions, like command substitution. We can invoke a command, like *date*, and then use its output as part of a file-naming scheme. We can even automate backups, and each copied file can have the value of the current date appended to the end of its name. Shell scripts are not just invocations of different commands either, but are programs in their own right. Shell scripting also allows us to use different programming functions—such as *if/then/else* statements, ‘for’ loops, and so on—directly within our operating system’s interface. We don’t have to learn another language because we are using what we already know—the command line.

Different features of shell scripts

Let’s have a look at the different features of shell scripts.

Running batch jobs: Shell scripts allow different sets of commands that are entered manually at a command line interface to be executed automatically, without having to wait for a user to separately trigger each stage of the sequence. For example, in a directory that has three C source



Figure 2: The different functions of a shell
(Image credits: googleimages.com)



Figure 3: Path shortcuts in shell scripting
(Image credits: googleimages.com)

code files, instead of manually running the four commands which are required to build the final program from them, one could create a C shell script, and keep it in the directory along with the C source code files, which would ultimately compile them automatically. The script would further allow a user to save the file being edited, pause the editor, and then just run the shell script to create the updated program, test that, and return to the editor again.

Different shortcuts to perform specific functions: We can use the different available shortcuts in shell scripting to perform some specific action. These shortcuts are short commands written for specific utilities. They are like inbuilt functions, which we use in different programming languages. A shell script can also provide convenient variations of a system command, where different special environment settings, post-processing or command options apply automatically, but in such a way that they allow the new script to still act as a complete normal UNIX command. For example, to create a version of the *ls* command to list different files, it can be given a shorter command name of ‘l’. This shorter command will be normally saved in the *bin* directory of a user as */home/username/bin/l* and a default set of different command options will be pre-supplied. The user can then use ‘l’ for most commonly used short listings.

Generalisation: Simple batch jobs are useful for isolated tasks, but at the same time, using shell loops, variables and tests provides much more flexibility to users. Let’s look at a bash file (shell script) to convert different JPEG images to PNG images, in which the names of images are provided on the command line—possibly using wildcards. Instead of each of the names being listed within the script, they can be created with this file, typically saved in */home/username/bin/jpg2png*. So we can directly run this command on an entire directory of JPEG images with just */home/username/bin/jpg2png *.jpg*.

Flavour of programming: Many modern shell scripts also supply various features which are usually found only in more sophisticated programming languages, such as control-flow constructs, comments, variables, subroutines, arrays and so on. With such features available, it is quite possible to write reasonably sophisticated applications using shell scripts. However, they are still limited by the fact that most shell scripting languages have little or even no support for data typing systems, threading, classes, complex math and other such common language features. They are also much slower than the compiled code or interpreted languages that are written with speed as a performance goal. The standard UNIX tools, Awk and Sed, provide extra capabilities for shell programming. Perl can also be embedded in different shell scripts as can other scripting languages such as Tcl, etc.

Verisimilitude: A key trait of different shell scripts is that the invocation of their interpreter is considered as a core operating system feature. So, instead of a user’s shell only being able to run different scripts in that shell’s language or a shell script only having directives of its